中山大学本科生期末考试

考试科目：《并行程序设计》（C 卷）

学年学期：**2022** 学年第 **2** 学期　　姓　　名：_____

开课单位：中山大学计算机学院　　学　　号：_____

考试方式：闭卷　　年　　级：_____

考试时长：**120** 分钟　　院　　系：_____

警示 《中山大学授予学士学位工作细则》第八条：“考试作弊者，不授予学士学位。”

----------以下为试题区域，共###道大题，总分 100 分,考生请在答题纸上作答----------

一、简答题（共 8 小题，每小题 5 分，共 40 分）

1. [5分] 简要说明单指令多数据流（SIMD）和单程序多数据流（SPMD）的区别与联系。
2. [5分] 什么是阿姆达尔定律（Amdahl’s law）？它告诉了我们什么？
3. [5分] 如何理解多线程编程中的线程安全性？如何解决线程安全性问题？
4. [5分] 简要说明线程间同步的方式（两种即可）。
5. [5分] CUDA编程中，可用于同步线程的机制有哪些？试举例说明其中三种。
6. [5分] 试说明原子操作的概念，并以一个CUDA中的原子操作函数为例，说明其在实际应用中的作用。
7. [5分] 简述在CUDA编程中，主机内存、设备内存、及统一内存寻址的概念，并说明统一内存寻址的作用。
8. [5分] 简述存储体冲突的概念，结合具体例子说明如何消除存储体冲突。

二、应用题（共 3 小题，每小题 5 分，共 15 分）

根据以下代码片段，回答以下有关于循环依赖与OpenMP调度的问题：

```
#pragma omp parallel for num_threads(4) schedule(static, 2)
    for (i = 0; i < N; i++) {
        A[i] = A[i] + B[i];
        B[i + 1] = C[i] + D[i];
    }
```

1.1 [5分] 由于循环依赖的存在，该并行代码无法产生期望的结果（与去掉OpenMP编译指导语句的串行版本一致的运行结果），请解释原因。

1.2 [5分] 若N = 16，请问各个线程分别计算数组A的哪些元素？

1.3 [5分]若N的值很大，且事先未知。实验发现，将schedule(static, 2)中的第二个参数trunk_size由2改为一个较大的值后（如10000），程序的性能有所提升。此时，还应修改原先的static调度方式以实现负载均衡。请问应将调度方式改为dynamic或guided中的哪一种？为什么？

三、编程题（共 1 小题，每小题 15 分，共 15 分）

现在有一个单精度浮点类型的数组A[N]，已知该数组的数据都分布范围为[0,10)。为了对这些数据的分布有一个更为直观的感受，可以画一个直方图：首先把这些数据的范围划分成10个同等大小的区间，也就是10个桶，然后统计每个桶中元素的数目。现在需要你编写一个MPI并行程序，实现函数histogram_gen，该函数的功能是完成上述过程，并将每个桶中元素的数目通过printf函数输出到标准输出。请注意，数组A保存在Rank 0中，你需要想办法把它分配到各个进程。

需要实现的函数原型：void histogram_gen(float* A, int N, MPI_Comm comm)

MPI 参考函数接口

函数	参数
int MPI_Init(int *argc, char **argv[])	参数就是main传进来那两个参数的地址
int MPI_Finalize(void)	
int MPI_Comm_size(MPI_Comm comm, int *size)	MPI_Comm comm 通信子;
	int size：返回进程的个数
int MPI_Comm_rank(MPI_Comm comm, int *rank)	MPI_Comm comm 通信子;
	int rank：返回进程的编号
int MPI_Barrier(MPI_Comm comm)	参数是通信子（常用MPI_COMM_WORLD）
int MPI_Send(void* msg_buf_p, int msg_size, MPI_Datatype msg_type, int dest, int tag, MPI_Comm communicator)	void* msg_buf_p： 发送缓冲区的起始地址;
	int buf_size： 缓冲区大小（该数据类型的个数）;
	MPI_Datatype msg_type： 发送信息的数据类型（如：MPI_CHAR、MPI_INT、MPI_DOUBLE）;
	int dest： 目标进程的id值;
	int tag： 消息标签，也可以是MPI_ANY_TAG;
	MPI_Comm communicator： 通信子;
int MPI_Recv(void* msg_buf_p, int buf_size, MPI_Datatype msg_type, int source, int tag, MPI_Comm communicator， MPI_Status *status_p)	void* msg_buf_p： 缓冲区的起始地址;
	int buf_size： 缓冲区大小（该数据类型的个数），要大于等于发送方的buf_size;
	MPI_Datatype msg_type： 发送信息的数据类型（如：MPI_CHAR）;
	int dest： 目标进程的id值，也可以是MPI_ANY_SOURCE;
	int tag： 消息标签，也可以是MPI_ANY_TAG;
	MPI_Comm communicator： 通信子;
	MPI_Status *status_p： status_p对象，包含实际接收到的消息的有关信息，默认为MPI_STATUS_IGNORE，被忽略
int MPI_Reduce(void * input_data_p, void * output_data_p, int count, MPI_Datatype datatype, MPI_Op operator, int dest_process, MPI_Comm comm)	void * input_data_p： 每个进程的待处理数据存放在input_data_p中;
	void * output_data_p： 存放最终结果的目标进程的地址;
	int count： 缓冲区中的数据个数;
	MPI_Datatype datatype： 数据项的类型;
	MPI_Op operator： 操作子**（如：MPI_SUM、MPI_MAX、MPI_MIN）;**
	int dest_process： 目标进程的编号;
int MPI_Allreduce(void * input_data_p, void * output_data_p, int count, MPI_Datatype datatype, MPI_Op operator, int dest_process, MPI_Comm comm)	void * input_data_p： 每个进程的待处理数据存放在input_data_p中;
	void * output_data_p： 存放最终结果的目标进程的地址;
	int count： 缓冲区中的数据个数;
	MPI_Datatype datatype： 数据项的类型;
	MPI_Op operator： 操作子（如：MPI_SUM、MPI_MAX、MPI_MIN）;
int MPI_Bcast(void* buffer, int count, MPI_Datatype datatype, int source, MPI_Comm comm)	void* buffer 缓冲区的起始地址，既是输入缓冲区，也是输出缓冲区;
	int count 缓冲区中的数据个数;
	MPI_Datatype datatype 缓冲区中的数据类型;
	int source 发送信息的进程id;
	MPI_Comm comm 通信子;
int MPI_Gather(void* sendbuf, int sendcount, MPI_Datatype sendtype, void* recvbuf, int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm)	void* sendbuf 发送缓冲区的起始地址
	int sendcount 发送缓冲区的数据个数
	MPI_Datatype sendtype 发送缓冲区的数据类型
	void* recvbuf 接收缓冲区的起始地址
	int recvcount 从每个进程接收的元素个数，应与recvtype一致
	MPI_Datatype recvtype 接收的数据类型
	int root 接收进程id
	MPI_Comm comm 通信子
int MPI_Allgather(void* sendbuf, int sendcount, MPI_Datatype sendtype, void* recvbuf, int recvcount, MPI_Datatype recvtype, MPI_Comm comm)	void* sendbuf 发送缓冲区的起始地址
	int sendcount 发送缓冲区的数据个数
	MPI_Datatype sendtype 发送缓冲区的数据类型
	void* recvbuf 接收缓冲区的起始地址
	int recvcount 从每个进程接收的元素个数，应与recvtype一致
	MPI_Datatype recvtype 接收的数据类型
	MPI_Comm comm 通信子
int MPI_Scatter(void* sendbuf, int sendcount, MPI_Datatype sendtype, void* recvbuf, int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm)	void* sendbuf 发送缓冲区的起始地址
	int sendcount 发送给每个进程的数据个数，应与recvtype一致
	MPI_Datatype sendtype 发送缓冲区中的数据类型
	void* recvbuf 接收缓冲区的起始地址
	int recvcount 待接收的元素个数
	MPI_Datatype recvtype 接收的数据类型，应与sendtype一致
	int root 发送进程id
	MPI_Comm comm 通信子
int MPI_Sendrecv(void* msg_buf_p, int msg_size, MPI_Datatype msg_type, int dest, int tag, void* msg_buf_p, int buf_size, MPI_Datatype msg_type, int source, int tag, MPI_Comm communicator， MPI_Status *status_p)	void* msg_buf_p： 发送缓冲区的起始地址;
	int buf_size： 缓冲区大小（该数据类型的个数）;
	MPI_Datatype msg_type： 发送信息的数据类型
	int dest： 目标进程的id值;
	int tag： 消息标签（真的就是一个标签，由用户指定，非负）;
	void* msg_buf_p： 缓冲区的起始地址;
	int buf_size： 缓冲区大小（该数据类型的个数），要大于等于发送方的buf_size;
	MPI_Datatype msg_type： 发送信息的数据类型（如：MPI_CHAR）;
	int dest： 目标进程的id值，也可以是MPI_ANY_SOURCE;
	int tag： 消息标签（真的就是一个标签，非负），也可以是MPI_ANY_TAG;
	MPI_Comm communicator： 通信子;
	MPI_Status *status_p： status_p对象，包含实际接收到的消息的有关信息，默认为MPI_STATUS_IGNORE，被忽略

四、CUDA代码分析（共 2 小题，每小题 15 分，共 30 分）

1. 线程执行分析：

```
__global__ void func(float* input, float* output, int size) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    if (tid * 2 > size) return;
    if (tid %2){
        output[tid] = input[tid*2]*input[tid*2]+input[tid*2+1]*input[tid*2+1];
    } else {
        output[tid] = input[tid*2]*input[tid*2]-input[tid*2+1]*input[tid*2+1];
    }
}
```

a) 说明以上代码功能。（4 分）

b) 说明其效率存在的问题，如何改进，写出改进后的源码。（11 分）

2. 代码分析：

a) 说明以下代码的功能；（5 分）

b) 说明其效率存在的问题，如何改进，写出改进后的源码；（10 分）

```
__global__ void func(int * out, int* in, int n, int m){
    int x = blockIdx.x*blockDim.x+threadIdx.x;
    int y = blockIdx.y*blockDim.y+threadIdx.y;
    out[x*n+y] = in[y*m+x];
}
```

排版要求：试卷一级标题须标注分值，至少占两行，字体加粗。行距适当留疏，格式整齐一致，保持卷面美观。各题之间不留答题区域，考生一律将答案写在专用答题纸（本）上。**试卷头版式不可改变，内容可根据考试科目实际需要增减**。

说明：此份仅为试卷模板，教师需根据实际内容将红色字迹替换。